

RAG: From Prototype to a Defensible System

Retrieval-augmented generation selects external evidence and provides it to a model. It can improve freshness and grounding, but does not automatically prevent hallucinations or enforce authorization. A credible system has versioned sources, measured retrieval, supported citations, and explicit abstention and failure behavior.

For: Developers building document-grounded assistants who want to move beyond a chat demo and explain ingestion, retrieval, permissions, evaluation, freshness, and recovery.

Suggested pace: 3-4 weeks with a small authorized corpus | APIs, embeddings, and ML evaluation basics

What you will be able to do

- Design a versioned ingestion and retrieval pipeline with traceable chunks.
- Evaluate retrieval separately from answer generation and citation support.
- Enforce tenant and document permissions before context reaches a model.
- Handle updates, deletions, latency budgets, and incomplete responses deliberately.

Build the right mental model

Chunking is a document decision

Preserve headings, table context, source IDs, versions, and permissions. Chunk size and overlap should be evaluated on representative queries, not chosen from one universal number. Small chunks can lose meaning; large chunks can dilute relevance and consume context.

Retrieval and generation fail differently

A wrong answer may result from missing evidence, poor ranking, incorrect filtering, truncated context, or unsupported generation. Inspect each stage. An aggregate answer score alone cannot tell you which component needs repair.

Authorization precedes generation

Apply tenant and permission constraints during retrieval where possible and recheck current authorization before building context. Retrieved text is evidence, not authority to call tools or change policy. Cache eligibility must include the relevant identity and version boundaries.

Study with evidence.

Start with a small corpus you are allowed to publish or process. Create answerable, unanswerable, ambiguous, updated, and access-restricted queries before tuning. Keep a deterministic retrieval baseline so improvements can be compared fairly.

Week 1: Define and ingest

Specify users, documents, freshness, permissions, and response behavior. Parse structure, assign stable chunk/source versions, and build a basic lexical or vector retrieval path. Keep raw source provenance available.

Deliverable and check: A small versioned corpus and an evaluation set with expected evidence, not just ideal answer wording.

Week 2: Improve retrieval

Compare lexical, dense, and hybrid retrieval, deduplication, filtering, and optional reranking. Measure recall and ranking on the same query set. Inspect failures before adjusting chunking or embedding models.

Deliverable and check: A retrieval report identifying which queries improved, regressed, or still lack evidence.

Week 3: Ground answers safely

Build a bounded context, require traceable citations, and define abstention when evidence is insufficient. Test permission revocation and misleading instructions in documents. Measure claim support rather than trusting a citation's presence.

Deliverable and check: A failure suite covering unsupported answers, stale sources, and cross-user access boundaries.

Week 4: Operate and explain

Add source updates, deletion propagation, model/prompt versions, latency and token budgets, cancellation, and sanitized telemetry. Present a project narrative with a baseline, evidence, limitations, and recovery behavior.

Deliverable and check: A reproducible demo whose claims are supported by evaluation rather than a few favorable examples.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Repair a stale-policy answer

A policy document changes from version 1 to version 2. The assistant still answers using an old chunk and a cached response. You need both immediate correctness at the read boundary and eventual cleanup of stale artifacts.

A worked approach

1. Give source documents and chunks explicit versions, and record which versions support each answer. On update, mark version 1 superseded in an authoritative document record before relying on all index replicas to finish updating.
2. Filter or recheck retrieved chunks against current source eligibility before prompt construction. Invalidate response caches whose source dependencies include the old version, or make version-aware eligibility reject them.
3. Re-index version 2, remove stale vectors and text according to retention policy, and rerun queries that distinguish the changed policy. Check citations and generated claims, not merely whether the new document appears somewhere in the index.

Expected result

New answers use eligible source versions even while background index cleanup is still converging. The system can explain provenance and avoid treating cache freshness as an accidental side effect of TTL.

Verify the result

- Can a stale replica still send a superseded private passage to the model?
- Does cache eligibility include source and permission context?
- Do update-specific queries verify both retrieval and final answer correctness?

Extend the exercise

Replace a content update with access revocation. Explain why final authorization must fail closed when current permission cannot be established, and why previously delivered text cannot be retroactively made unread.

Make the learning stick.

Does RAG eliminate hallucinations?

No. Retrieval can miss evidence, context can be misused, and generation can add unsupported claims. Evaluate groundedness and abstention explicitly, and keep a way to inspect which source passages were actually supplied.

Can a citation prove an answer is supported?

No. The cited passage may be irrelevant, outdated, or not support the specific claim. Validate claim-to-evidence alignment and use human review for important or uncertain cases.

Should every RAG application become an agent?

No. A deterministic retrieval-and-answer pipeline may be easier to evaluate and operate. Add dynamic tool use only when the task needs it and the permissions, budgets, and recovery behavior are designed explicitly.

Avoid these traps

- Tuning on a few favorite demo questions.
- Sending unauthorized passages to a model and filtering only afterward.
- Treating source updates and deletions as optional index maintenance.

Ready for the next step?

- Sources, chunks, prompts, and models are versioned.
- Retrieval and answer quality are evaluated separately.
- Permissions are enforced before context construction.
- Freshness, abstention, and recovery paths are tested.

Reliable next reads

- [Azure RAG design and evaluation](#)
- [Google embeddings introduction](#)
- [OWASP GenAI Security Project](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.